

LARS JANSEN

Senior Software Engineer

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Full-stack engineer and pragmatic architect with 10+ years building reliable, data-heavy web platforms. For most of my career, my mix has been **Angular + TypeScript** on the front end, **.NET / ASP.NET Core + SQL** on the back end, and **AWS/Azure** for cloud delivery. I ship clean, observable APIs and scalable UIs: Angular for business workflows, .NET services for core logic and integrations, and cloud automation for secure, repeatable releases. I use LLM tools to speed up testing, refactors, and documentation with strict guardrails (no secrets, no PII, human review first).

SKILLS

- **Frontend (Angular)** — AngularJS 1.5 (component pattern), **Angular 2→18**, React, TypeScript, RxJS 5→7, **NgRx** (Store/Effects/Entity), Angular Material & CDK, **Standalone components** (v14+), **Signals** (v16+), new control flow (v17+), **Angular Universal** SSR/hydration (v17+), Nx monorepos; unit/e2e with Jasmine/Karma or Jest + **Testing Library** and **Playwright**.
- **Backend** — **.NET Framework (4.6–4.8)**, **.NET (6–8)**, **C#**, **ASP.NET Core**, **ASP.NET MVC / Web API**, **Swift**, **C++**, Minimal APIs, **EF Core**, **Dapper**, **LINQ**, SQL Server, PostgreSQL, Redis, **gRPC**, **SignalR**, FluentValidation, Swagger/OpenAPI (Swashbuckle), auth with **OAuth2/OIDC**, **JWT**, **RBAC/ABAC**, rate limiting, background processing (**IHostedService/BackgroundService**), Hangfire.
- **Cloud & DevOps** — **Docker**, **Kubernetes**, Helm, Terraform, GitHub Actions, Azure DevOps Pipelines, **AWS** (ECS, EKS, Lambda, API Gateway, S3, RDS, CloudWatch), **Azure** (App Service, AKS, Azure SQL, Key Vault, Application Insights), blue/green + canary releases, IaC-backed environments, secrets management, CI/CD quality gates.
- **Observability & Quality** — **OpenTelemetry**, Serilog, ELK/EFK, Prometheus, Grafana, Datadog APM, structured logging, correlation IDs, distributed tracing, SLO-driven alerting, contract testing (Pact), performance/load testing (k6).

WORK HISTORY

Senior Software Engineer
RavenTrack

August 2021 to November 2025

- In a premium fashion e-commerce program, led **.NET 8 (ASP.NET Core Web API)** and **Angular 18 + TypeScript** delivery for catalog, checkout, and account flows under strict **SLA** expectations.
- Designed **REST** contracts with **OpenAPI/Swagger** and versioned endpoints, so mobile/web clients could ship independently while phasing out breaking changes safely.
- Upgraded a legacy backend from **.NET Framework 4.8 / .NET Core 3.1** patterns into **.NET 6** and later **.NET 8**, resolving **DI lifetime leaks**, obsolete middleware, and async deadlocks during the cutover.
- **Leveraged over 5+ years of expertise in C# and .NET Core** to design and maintain cross-platform desktop applications with a strong focus on **macOS architecture**, optimizing application performance, resolving complex runtime and UI rendering issues, and ensuring smooth interoperability between **macOS and Windows environments**.
- Utilized **Swift and C++** to build native macOS components and integrate them with .NET-based systems, demonstrating a deep understanding of **macOS and Windows system architectures**, while also contributing to legacy tooling where **Delphi** was optionally used for maintaining compatibility with older desktop modules.
- Fixed a critical production bug where currency rounding and locale parsing caused silent penny drift in totals, by enforcing **decimal math**, invariant formatting, and regression tests for edge-case baskets.
- Resolved a high-impact auth incident where users hit infinite loading due to a race between token refresh and guarded routes, by adding a single refresh **mutex** in an Angular **HttpInterceptor** with retry rules.
- Implemented a new feature for real-time order status using **SignalR** and optimistic UI, including reconnect backoff, message versioning, and graceful fallbacks to polling when streams degraded.
- Rebuilt state handling with **NgRx 18** selectors and immutable updates, which removed ad-hoc subscriptions and eliminated stale UI bugs caused by partial API payload merges.
- Optimized API latency by introducing **Redis** caching with explicit TTL and cache-bust rules, and added **idempotency keys** on write endpoints to prevent duplicate charges on retries.
- Hardened background processing with **Hangfire** and queue-driven workers, implementing **DLQ** reprocessing, poison-message quarantines, and idempotent handlers to stop double-fulfillment incidents.
- Handled a severe data bug where a timezone conversion shifted delivery promises by one day for some regions, by normalizing timestamps to **UTC** at ingestion and standardizing serialization end-to-end.
- Introduced **CI/CD** using **Azure DevOps** pipelines, gated merges with unit/integration tests, and added automated smoke tests to detect config drift across environments before customer impact.
- Improved reliability with structured logs, correlation IDs, and **OpenTelemetry** traces flowing into **Application Insights**, making root-cause analysis fast across API calls, jobs, and UI actions.
- Implemented rate limiting and abuse protection using middleware and **JWT** claims, adding audit logs and **RBAC** checks so customer support tools stayed secure without slowing down operations.
- Modernized UI performance by profiling change detection, applying **OnPush**, **trackBy**, route-level lazy loading, and bundle budgeting, which reduced first load and prevented regressions in key flows.
- Stayed flexible across frontend and backend by pairing Angular component design with .NET domain modeling, unblocking releases during outages while coaching teammates on safer patterns and migrations.

Software Engineer

June 2018 to July 2021

Infinite Loop

- Delivered **B2B SaaS** features for a retail / e-commerce operations platform using **C#, ASP.NET Core 3.1**, and **Angular 11** with **TypeScript 4.1**, keeping releases stable under weekly change pressure.
- Designed **REST APIs** with **OpenAPI/Swagger**, consistent error envelopes, and strict request validation, so Angular clients could evolve safely without breaking contracts across multiple squads.
- Fixed a critical production bug where users saw duplicated orders after timeouts by adding **idempotency keys**, transactional guards in **EF Core 3.1/5**, and replay-safe handlers for retried requests.
- Resolved a high-impact **timezone** bug where “next-day” delivery windows shifted for some locales by normalizing all timestamps to **UTC**, enforcing **ISO-8601** serialization, and backfilling corrupted rows with audited scripts.
- Upgraded a legacy AngularJS-style module into a modern **Angular 11** feature shell with lazy routing, **RxJS 6** streams, and typed reactive forms, reducing regressions caused by ad-hoc subscriptions and shared state.
- Implemented a new real-time dashboard feature using **SignalR** and resilient reconnection logic, including message versioning and UI fallbacks, so operations teams had live status without full page refreshes.
- Handled a severe **memory leak** in the UI caused by orphaned subscriptions by standardizing **async pipe** usage, introducing facade services, and adding component teardown tests to prevent recurrence.
- Improved API performance by analyzing SQL plans on **SQL Server 2017/2019**, adding covering indexes, rewriting N+1 query patterns, and introducing server-side pagination to stabilize p95 latency under peak loads.
- Implemented **JWT** authentication with role-based authorization policies and audit-friendly claims mapping, then added rate limiting and structured security logs to reduce brute-force noise and speed up investigations.
- Modernized background processing by migrating cron-like tasks into **IHostedService** workers with retries, backoff, and dead-letter patterns, which reduced stuck jobs and made recovery deterministic after outages.
- Built **CI/CD** pipelines in **Azure DevOps** with gated builds, database migration checks, and environment-based configuration, ensuring every deployment had consistent artifacts and reliable rollback paths.
- Introduced **Docker** for repeatable local and CI environments, aligning Angular build outputs and .NET runtime dependencies so “works on my machine” incidents stopped blocking releases.
- Fixed a critical rounding and currency formatting defect that silently changed invoice totals by switching to decimal-safe calculations, invariant culture formatting, and regression tests over edge-case inputs.
- Implemented a forward-looking **feature flag** system for incremental rollout, allowing the team to ship risky UI refactors safely while collecting metrics and reverting instantly when anomalies appeared.
- Used skills flexibly across frontend, API, and data layers by pairing with QA and product on incident triage, then translating findings into code-level fixes, monitoring alerts, and prevention playbooks.

Software Developer Datamart

October 2015 to May 2018

- Delivered customer-facing workflows in **Angular 4/5** with **TypeScript 2.x**, **RxJS 5**, reactive forms, and route guards, keeping UI state predictable and tightly aligned with **ASP.NET MVC 5 / Web API 2** contracts for clean QA sign-off.
- Built and maintained **.NET Framework 4.6/4.7** services with shared validation filters and consistent error envelopes, reducing integration ambiguity so the Angular client stopped breaking on edge-case inputs from real users.
- Supported performance recoveries by profiling slow grids and report screens, then applying server-side paging and search in **SQL Server 2014/2016** with query-plan reviews and targeted indexes to stabilize response times under peak usage.
- Helped fix a critical production bug where double-click submissions produced duplicate records and occasional double charges, adding API-side **idempotency tokens**, **optimistic concurrency** checks, and UI-side disabled submit states plus safe retry messaging.
- Assisted with “random logout” incidents caused by token refresh races by consolidating the **Angular HTTP interceptor** flow and aligning session timeout rules in API middleware so authentication stayed consistent across tabs and refresh cycles.
- Implemented supportable background processing for long-running operations using **Windows Services** and scheduled jobs with durable tracking tables, enabling safe retries and admin re-runs without corrupting business data.
- Upgraded legacy pages built with older Angular patterns and mixed jQuery widgets into a cleaner **Angular 5** feature-module structure, introducing shared components, stricter typing, and consistent form validators to reduce regressions.
- Contributed to backend cleanup by separating controller concerns from domain logic using **Entity Framework 6** service boundaries, which improved testability and reduced unintended side effects during urgent hotfixes.
- Added **structured logging** with correlation IDs across UI and API requests, making production-only failures traceable in multi-step workflows where timing and partial retries previously hid the real root cause.
- Improved release reliability by strengthening **CI** build gates (MSBuild/NuGet restore), adding smoke tests for key API routes, and validating schema changes via staged migration scripts in pre-production environments.
- Worked closely with QA and product to convert unclear requirements into acceptance criteria, then delivered incremental changes behind **feature flags** so risky paths could be disabled quickly when unexpected edge cases surfaced.

EDUCATION

Bachelor's degree in Computer Science University of Amsterdam

2011 to 2015

- Built strong foundations in software engineering, data structures, and systems design, with coursework that supports designing reliable services and data-driven applications.

PROJECTS

Fashion Order Management & Returns Portal (Angular + .NET Modernization)

- I delivered a customer-facing **Angular** (TypeScript) portal for order tracking and returns, integrating with **ASP.NET Web API** services to keep checkout, refunds, and status updates consistent across web and support tooling.
- I fixed a critical bug where rapid double-click submissions created duplicate RMAs and double refunds, solving it with server-side **idempotency keys**, database uniqueness constraints, and a UI “single-flight” submit pattern with clear retry messaging.
- I upgraded a legacy frontend area from **AngularJS 1.x** to **Angular 5** by carving out feature modules, rewriting shared components, and aligning build tooling so the team could ship new screens without breaking older flows.
- I improved reliability and performance by adding SQL indexes for high-traffic queries, implementing server-side pagination, and introducing structured logging with correlation IDs so production incidents could be traced across UI, API, and database layers quickly.